

Analysis: Datacenter Duplex

We can start by modeling the problem as a graph where each cell of the input matrix represents a node. Orthogonally adjacent cells with the same label are connected by edges, and we can optionally add edges connecting diagonally adjacent cells with the same label. The goal is to end up with exactly two [connected components](#).

Test set 1

Test set 1 can be solved with [dynamic programming](#), iterating over columns. When considering the i -th column from left to right, we can summarize the state of the connections we have added in the submatrix S of columns 1 through i by recording: (1) whether we have seen (at least one cell of) each of A and B so far, and (2) whether any two cells on the i -th column that contain the same label but are not orthogonally adjacent are connected by a path in S . Then, we can use brute force, and consider all possible choices of how to connect each of the up to 3 cell corners between columns i and $i+1$ (there are at most 4 rows in Test set 1). There are 3 possibilities for each corner, $\backslash$, $/$ and no connection, but it can be reduced by noticing if at least one of the connections is valid, it is always optimal to use one, which reduces the number of choices per corner to 2 at the most. If at any point we discover a new isolated A component that cannot be connected to previously seen A s, we have failed, and same is true for B . If we finish, we can then use a second pass to reconstruct one choice of diagonal connections that led to solving the problem.

This seemingly simple idea has several technical details that we are omitting here. Some of them can be simplified by starting and ending the process in columns that contain both A s and B s and preprocessing to check if any leftmost or rightmost columns that contain only one type already disconnect the other type.

The overall time complexity of this solution is exponential in R , because we are trying all combinations of $O(R)$ corners and recording the connected status of $O(R)$ cells at each state, and linear in C , with the exact formula depending on how the technical details are handled. As long as the base of the exponential factor is not too large, this is fast enough to pass within the time limit.

Test set 2

The key observation is: after we decide to add some edges (diagonal adjacencies), two cells c and d containing the same label X end up separated regardless of any future decisions if and only if one of the following two conditions holds:

- 1. There is a cycle of cells labeled $Y \neq X$ in the graph, with one of c and d being inside the cycle and the other one being outside.
- 2. There is a path of cells labeled $Y \neq X$ in the graph with the first and last cells of the path being border cells of the matrix, with c and d being on opposite sides of the path.

Let us use G to denote the graph with no diagonal edges added and H to denote the final graph with all edges added. Consider the border of the matrix. Suppose that it contains two cells c and d labeled X that are not connected in G . Since they are not connected in G , that means, going around the border, there are cells e and f labeled $Y \neq X$ such that e is in between c and d in clockwise order and f is between c and d in counter-clockwise order. That means neither c and d nor e and f can be connected in H with a path of only border cells. Therefore, if c and d are connected in H , the path connecting them disconnects e and f , and vice versa. So, by the

second condition, having two cells on the border with the same label that are disconnected in G results in an impossible case.

Notice that there is never an incentive to generate an edge from a diagonal adjacency to connect two cells that are already connected. Per the paragraph above, if a case is possible, then all border cells with the same label are already connected in G . Therefore, if an algorithm never adds an edge from a diagonal adjacency that connects two cells that are already connected, it will never generate a path between two border cells. Additionally, if we never connect cells that are already connected, any cycle in H is a cycle that was already present in G , so again, if we end up in a disconnected situation, the case must have been impossible from the start, before we added any connections.

These observations directly suggest the following algorithm: consider each diagonal adjacency and generate an edge if and only if it will connect two previously disconnected cells. If both choices work, we can choose either, since we already established that the decision of not connecting previously connected cells is enough to guarantee an algorithm will not generate an H with more than 2 connected components when a different one with exactly 2 was possible. After this process, check if there are 2 connected components or more than 2, and print the results.

If we implement the algorithm above using a [union-find](#) to maintain the connected components, we need $O(\mathbf{R} \times \mathbf{C})$ checks for whether two cells are connected and $O(\mathbf{R} \times \mathbf{C})$ connections (joins). Since union-find provides almost constant amortized time for both operations, the overall time complexity of the algorithm is quasilinear.

An equivalent algorithm is to calculate the connected components of G first and then use shortest paths to join any two components of the same label until we cannot do it anymore. Since shortest paths cannot create new cycles, an argument similar to the one above proves that this solution is also correct. This solution can be implemented in linear time if the partial minimum path tree graph is reused for each new connection we need.

Analysis: Napkin Folding

Test Set 1

For Test Set 1, we want to find exactly one folding segment such that when we fold the napkin across the segment, the regions on either side of the segment line up perfectly. Because the folding segment must split the napkin into two symmetric regions, we can show that each of the folding segment's endpoints either coincides with a vertex of the polygon defining the napkin, or is the midpoint of an edge of the polygon. If we tried to make a folding segment connecting any other points, the two parts of the split edge could not possibly line up perfectly. Thus, all we need to do is try every line segment connecting any pair of points that are vertices or midpoints of the polygon's edges. Then, for each potential folding segment, we check whether it is valid by making sure it is fully contained within the polygon and that the two regions it creates are symmetric across the folding line segment.

[Checking for intersections between line segments](#) can be done by using only integers. [Reflecting points across a line](#) or taking the midpoint of a side normally could produce points with non-integer coordinates. But we can scale up our points initially such that if the reflection were to produce a point with non-integer coordinates, it could not possibly be one of the valid endpoints for folding line segments. Of course, we can also choose to work with fractions.

With N points in our polygon, we have $2 \times N$ points to choose from for our folding segment's endpoints. Each potential folding segment can be checked in $O(N)$ time. Because there are $O(N^2)$ possible segments to check, this results in an $O(N^3)$ overall time complexity.

Note that in order to check for symmetry across a folding segment, we cannot just check that the sets of vertices of the polygons defining each region are symmetrical. Rather, we must show that for every edge of the polygon of one region, there exists exactly one edge in the polygon defining the second region which is symmetric across the folding line segment. In other words, the order in which the points appear on each side matters.

Finally, we can note that if a given line is indeed an axis of symmetry of the polygon, then it is guaranteed that it doesn't intersect the polygon more than twice. This means that we don't need an explicit check in the code for the folding segment not to intersect the polygon outside its endpoints. A similar simplification can be applied to the solution of Test set 2, coming up next.

Test Set 2

Since we want to draw a neat folding pattern of $K-1$ non-intersecting line segments, we must partition our napkin into K regions of identical size and shape, all of which are symmetric with other regions sharing common line segments. Each of these K regions is a polygon with edges that are made up of folding line segments and/or edges or parts of edges from the polygon defining the napkin. It can be shown that at least two of these regions are only adjacent to one line segment in our folding pattern, with the other edges that define the region's polygon coming from the original polygon. Let's call these regions *corner regions*.

If we have a corner region, we can reflect that region across its one folding line segment to find the region that must be adjacent to it. If we keep reflecting these regions across the edges of their polygons, being careful not to create intersecting regions or leave the polygon defining the napkin, we can reconstruct the neat folding pattern. Thus, every neat folding pattern can be uniquely defined by just one folding line segment connecting two points on the border of the

polygon defining the napkin. That segment cuts off a corner region which can be successively reflected to give us the entire folding pattern.

We need to consider pairs of points on the napkin's border defining a folding line segment. For a given candidate folding line segment, we can successively reflect the corner region we form to get the full folding pattern. If we use more than $K-1$ reflections, or, if after we finish reflecting we don't end up creating the original polygon, we know that the chosen line segment does not give us a neat folding pattern.

Now all that remains is to select all pairs of points on the napkin's border. Clearly we cannot test every pair of points with rational coordinates, because there are infinitely many such points. Rather, we can show that the endpoints of the line segments in our neat folding pattern must be either vertices or points on the polygon's edges that are X/Y of the way between consecutive vertices, where $1 \leq X \leq Y-1$ and $2 \leq Y \leq K$ (the proof for this is below). Therefore, we can create all of these candidate points and check every pair. With $K \leq 10$ and $N \leq 200$, there are at most 6400 candidate points for the vertices of the line segments in our neat folding pattern. This puts an upper bound of 6400^2 on the number of pairs that we might need to check. But, we can reduce this significantly by only considering pairs which create a corner region with an [area](#) equal to $1/K$ of the napkin's area. Every point can pair with at most 2 other points to create a line segment which is fully within the polygon and splits off a corner region with the proper area. Therefore, we only need to check these pairs.

We can check if a single line segment from a pair of points gives us a valid folding pattern in $O(N)$ time if we are careful to stop once a reflection creates a point which is not on the border of one of our polygon's line segments. We can precompute all the valid points and use a [hash table](#) for this check. Because all of the points are of the form $X/Y \times p$, where p is a point with integer coordinates and Y is between 2 and K , we can multiply everything by $\text{lcm}(2, 3, \dots, K)$ at the beginning and then work in integers, dividing and simplifying fractions only for the output.

To summarize, the algorithm requires these steps:

- 1. Compute all possible endpoints. ($O(N \times K^2)$)
- 2. Find candidate segments to create a corner region. ($O(N \times K^2)$).
- 3. For each candidate, fold it up to $K-1$ times, checking that everything is valid.

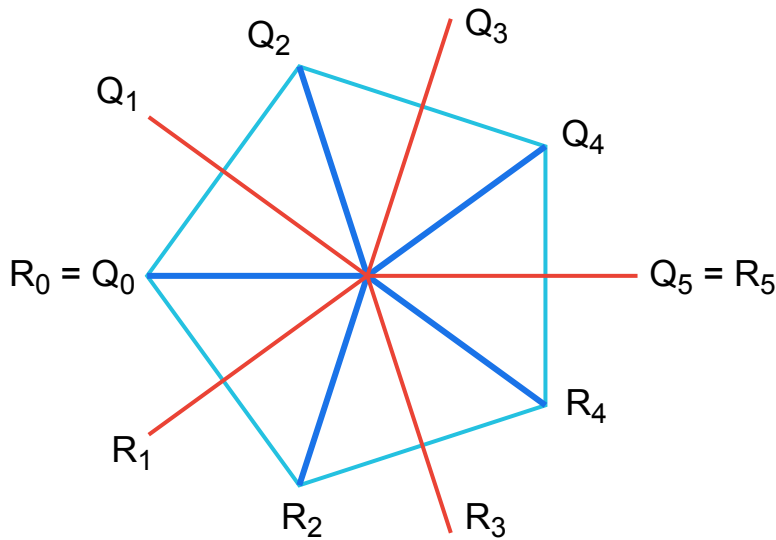
There are up to $O(N \times K^2)$ possible endpoints. If we fix one endpoint P and iterate the other, we can compute the area as we advance, finding the up to 2 segments that have an endpoint in P in $O(N \times K^2)$ time. So step 2 takes $O(N^2 \times K^4)$ time in total. For step 3, each unfolding requires reflecting the current region and finding new folding segments. All of that is linear in the size of the region, and the sum over the sizes of all regions is at most the total number of endpoints plus $2 \times K$ shared ones, so this takes $O(N \times K^2)$ time overall. Putting it all together, the algorithm takes $O(N^2 \times K^4)$ time in total. It is possible to make it significantly more efficient than that, but the implementation is complicated enough as it is.

Appendix

To prove that all folding segments endpoints are X/Y of the way between consecutive vertices for $1 \leq X \leq Y-1$ and $2 \leq Y \leq K$, we can prove that the number of folding segments that are incident on a non-vertex point of the input polygon is odd or zero. If that's true, let P be an endpoint and Q and R be the two vertices and/or folding segment endpoints that are closest to P towards each side over the same polygon edge E as P . Then, by reflection PQ and PR are equal. By repeating this over E we can see that E is divided into equal length sections by every point that is a folding segment endpoint.

Now we prove that the number of incident folding segments in a non-vertex point of the polygon is odd or zero. Assume that P is a point over an edge E of the polygon with $l > 1$ of incident folding segments. Let Q_i for $i = 1, 2, \dots, l$ be the non- P endpoints of each of those segments, in clockwise order. Let Q_0 be the reflection of Q_2 across PQ_1 and Q_{l+1} the reflection of Q_{l-1} across PQ_l . Notice both those points have to be on E . Now, the $l+1$ angles $Q_i Q_{i+1}$ for $i = 0, 1, \dots, l$ are all equal. Let R_i be the reflection of Q_i across E . Now, because Q_i must reflect onto Q_{i+2} , the length of $PQ_0, PQ_2, PQ_4, \dots, PQ_l$ are all equal, and equal to the lengths of $PR_2, PR_4, \dots, PR_l$. Since the angles between two consecutive segments of those are also all equal, $Q_0 Q_2 Q_4 \dots Q_l R_l R_{l-2} \dots R_2$ is a regular polygon. All points have rational coordinates because they are either input points, endpoints of folding segments, or reflections calculated from other points with rational coordinates. It is known that the only regular polygon whose vertices have rational coordinates in 2 dimensions is the square, which means $l/2 + 1 + l/2 = 4$, which implies $l = 3$ is odd.

The following picture illustrates the above proof for the case $l = 4$. P is the point in the center, and line segments of the same color are guaranteed to be of the same length.



A consequence of this proof is that for any non-vertex point on the original polygon, it must be adjacent to exactly 0, 1 or 3 folding line segments.

Analysis: Pancake Pyramid

We can make a couple of useful observations at the outset. First, if we have an interval of length 1 or 2, we do not need to add any pancakes for it to have the pyramid property, so we can ignore the restriction of length ≥ 3 in the problem. Second, for any interval, the "peak" in the optimal answer is the largest stack in the original interval (we leave this for you to think about). If there are multiple largest stacks in an interval, we will take the leftmost largest stack as the peak.

$O(S^3)$ — Too slow

For each of the $(S + 1)$ choose 2 intervals, determine where the peak will be located once the interval is turned into a pyramid. Once we know where the peak will be, we have two smaller problems: we need a non-decreasing sequence to the peak's left and a non-increasing sequence to the peak's right. To compute how many pancakes we need in the non-decreasing interval, we may simply sweep from the leftmost point and add pancakes until no stack in the interval to the left of the i -th stack is strictly taller than the i -th stack. By maintaining the running maximum as we sweep, we can compute the number of needed pancakes needed per interval in $O(S)$ operations. Since there are $O(S^2)$ intervals, this algorithm requires $O(S^3)$ operations in total.

$O(S^2)$ — Test Set 1

The ideas above lay the framework for a quicker solution. Instead of independently recomputing the number of pancakes needed to make an interval non-decreasing (or non-increasing), we can use the results from other intervals. Say we know the index of largest stack of pancakes in the range $[L, R]$ (call this index $M[L, R]$) and the smallest number of pancakes needed to make the interval $[L, R]$ into a non-decreasing sequence (call this number $X[L, R]$). We can compute both $M[L, R+1]$ and $X[L, R+1]$ in $O(1)$ time since the height at $M[L, R+1] = \max(P_{M[L, R]}, P_{R+1})$ and $X[L, R+1] = X[L, R] + (P_{R+1} - P_{M[L, R+1]})$. Similarly, we can store $Y[L, R]$, which is the smallest number of pancakes needed to make the interval $[L, R]$ into a non-increasing sequence.

For any interval $[L, R]$, the smallest number of pancakes needed to turn the interval into a pyramid is simply $X[L, M[L, R]] + Y[M[L, R], R]$. The precomputation takes $O(S^2)$ time and memory and the second step uses $O(1)$ time per interval to compute the answer. Thus, in total, this is $O(S^2)$.

$O(S \log S)$ — Test Set 2

The above strategy will be too slow and require too much memory to handle the larger bounds. For this test set, we will still use the same underlying idea of needing the number of pancakes to make an interval non-decreasing or non-increasing. But instead of computing X and Y , we will compute cumulative values: define $X'[L, R] = X[L, R] + X[L+1, R] + \dots + X[R, R]$ and $Y'[L, R] = Y[L, L] + Y[L, L+1] + \dots + Y[L, R]$. Now, instead of focusing on the left and right endpoints, we will base our strategy on the peaks of the intervals.

Initially, we do not know the value of X' or Y' for any interval. In our analysis, we will assume that we only know the X' and Y' values for maximal intervals. That is, if two intervals are side-by-side, we will merge them (we will never have intersecting intervals). For example, if we know

$X'[L, k]$ and $X'[k+1, R]$, we will merge these together into $X'[L, R]$ and forget about $X'[L, k]$ and $X'[k+1, R]$. The full process of how to merge is explained below. In particular, this means that any given stack is in at most one known interval for X' and one known interval for Y' .

We will process the peaks from smallest to largest. When we process the i -th stack, we are only interested in intervals that have stack i as their peak. If $X'[L, i-1]$ is computed for some value of L , then L must be the furthest left index such that $P_L, P_{L+1}, \dots, P_{i-1}$ are all less than P_i (since we are processing the stacks from smallest to largest). Similarly, if $Y'[i+1, R]$ is computed for some value R , then R must be the furthest right index such that $P_{i+1}, P_{i+2}, \dots, P_R$ are all at least P_i . If such L and R exist, then we can compute the number of pancakes needed over all intervals that have i as their peak:

$$X'[L, i-1] * (R - i + 1) + Y'[i+1, R] * (i - L + 1)$$

Note that if we don't know $X'[L, i-1]$ for any L , then $P_{i-1} \geq P_i$, so i cannot be a peak with any interval that includes both $i-1$ and i . The answer for those intervals will be computed later when we consider stack $i-1$ as the peak (and likewise for stack $i+1$ if we do not know $Y'[i+1, R]$ for any R). In these cases, we may use $X' = 0$ (or $Y' = 0$). Note that since our intervals are maximal and we are computing from smallest to largest, $P_{L-1} \geq P_i$ (similarly, $P_{R+1} > P_i$).

Now we want to merge $X'[L, i-1]$, $X'[i+1, R]$ and stack i into $X'[L, R]$. We will do this in two steps. First, note that $X'[L, i] = X'[L, i-1]$: since we are processing the stacks from smallest to largest, P_i can be freely added as the right endpoint of any non-decreasing sequence in this range. Now let's merge $X'[L, i]$ and $X'[i+1, R]$. Observe that $X'[L, R]$ sums over intervals that end at the R -th stack. If an interval starts in the range $[i+1, R]$, then it is already counted in $X'[i+1, R]$. If an interval starts in $[L, i]$, then we can start with some sequence in $[L, i]$, but since P_i is the peak, every value on the right must be exactly P_i . The number of pancakes needed to bring every value in $[i+1, R]$ up to P_i can be computed in $O(1)$ time using cumulative sums. Thus, the full merge is:

$$X'[L, R] = X'[i+1, R] + (X'[L, i-1] + P_i \times (i-L+1) \times (P_{i+1} + \dots + P_R))$$

The Y' values can be computed similarly. In terms of complexity, we need to sort the stacks at the beginning in $O(S \log S)$ time and the remaining steps take constant time per peak, so $O(S)$ overall. This means the algorithm takes $O(S \log S)$ time.

$O(S)$

Although the $O(S \log S)$ solution is fast enough to solve test set 2, an $O(S)$ solution is also possible! We present a sketch of the idea here, which can be read independently of the solution above.

To make things easier, we pretend that the stacks all have different heights. Each time we compare the heights of two stacks of identical height, we break the tie by assuming that the stack with the larger index is higher.

Let us think through the solution starting from the end. For each stack, we want to compute the pyramidification cost for all the ranges in which this stack is the highest; in such cases, it will be the peak of the pyramid. Then we can sum all of those values for all of the stacks, and that will be our overall result.

In order to compute those pyramidification costs, we can compute the following for each stack s :

- The nearest higher stack to the left of s (possibly an infinitely high "guard" stack appended to the beginning of the sequence); call this the "left blocker". Let D_L be the absolute distance (in stacks) from s to the left blocker.

- The nearest higher stack to the right of s (or guard stack) stack added after the sequence); call this "right blocker". Let D_R be the absolute distance (in stacks) from s to the right blocker.
- The pyramidification cost for all the ranges that end with s and in which s is the highest; call this "left pyramidification cost" C_L .
- The pyramidification cost for all the ranges that start with s and in which s is the highest; call this "right pyramidification cost" C_R .

Then the pyramidification cost for s can be calculated as $C_L \times D_R + C_R \times D_L$.

Now, how can we compute C_L and D_L for each stack? (The solution is analogous for C_R and D_R ; the only major difference is that the inequality used to compare stacks is strict on one side and non-strict on the other, due to tie-breaking.)

We can traverse the stacks from left to right, keeping a [Stack](#) structure (capital S to avoid confusion) X that remembers the longest decreasing sequence of stacks ending on the current stack. We iterate through the stacks and when seeing a stack s we consume from X all stacks that are lower than s , adding their contributions to the current left pyramidification cost. Let $t' = s$ at the beginning, and later $t' =$ the previously consumed stack. The contribution of a consumed lower stack t is the number of pancakes missing between t and t' (calculated in constant time, if we precompute the cumulative sum of stack sizes) multiplied by the distance to the left blocker of t' . The left blocker of s is the first stack we can't consume from X , because it's higher than s . Once we're done with s , we add s to X and keep going.

Analysis: Zillionim

There may not be a *zillion* possible solutions to this problem, but there are more than we can discuss in this analysis! We'll only touch on a few here, some of which work, and some of which don't. You're welcome to share yours in the [Code Jam Google Group](#)!

Random play

What if we always choose a move uniformly at random from among all available moves, just like the AI does? Intuitively, since the overall number of coins is large relative to the size of any one move, going first or second is not very important (given that in this case, both sides are playing randomly and not using any strategy), and every game is close to a fair coin flip. Suppose that we have a 50% chance of winning a game with this strategy; then, per the binomial distribution, we have only about an 0.0004% chance of winning 300 or more games out of 500. So this approach won't even get us the 1 point for test set 1!

A mirror strategy

If we were allowed to go first in this game, we could guarantee a win using the following strategy. Imagine a "center line" drawn between the $(10^{12} / 2)$ -th and $((10^{12} / 2) + 1)$ -th coins, dividing the coins into two regions of equal size. On the first turn, we could take the group of 10^{10} coins bisected by this line — that is, the coins numbered from $((10^{12} - 10^{10}) / 2) + 1$ to $((10^{12} + 10^{10}) / 2)$. Then, after every move by the opponent, we could make the same move, but reflected across this center line. For example, if the opponent were to take the 10^{10} coins starting from coin number 2, we would respond by taking the group of 10^{10} coins starting from the next-to-last coin and going left. We would always be guaranteed a move, by symmetry, so the opponent would have to eventually lose.

Sadly, we cannot move first, but we can try to adapt this strategy by just making the mirrored version of the opponent's move. This will always be possible unless the opponent makes a move that crosses the center line, in which case we cannot mirror it. But if the opponent happens to move close to the center line without crossing it — specifically, taking a coin fewer than $10^{10} / 2$ coins away from the center without crossing the center — then they will ensure their own defeat. If they do happen to make a move that crosses the center line, we can abandon our strategy and move randomly.

We can run a local simulation and find that this strategy does better than randomness, winning around 57.5% of the time. Unfortunately, this only gives us about a 14% chance of passing test set 1. Moreover, because our moves depend on the judge's moves, we cannot easily tweak the judge's randomness; we only get some control over that once the judge has made a move that hurts our strategy. So, all in all, it's probably best to abandon this approach.

A 2×10^{10} strategy

Let's call a remaining set of (at least 10^{10}) contiguous coins a "run". Observe that a "run" of exactly 2×10^{10} remaining coins can be very useful for us. We have the option of taking the first 10^{10} or the last 10^{10} coins from that run, and leaving a run of exactly 10^{10} coins (and therefore one possible move) behind. On the other hand, if we take any other group of 10^{10} coins from that run, we will leave no moves behind, "destroying" the run. Also notice that if the AI happens

to move within this run, it will virtually always make the second type of move; the odds of it happening to choose the first or last 10^{10} coins are negligible.

Because of this, a run of exactly 2×10^{10} coins lets us control the parity of the game. Suppose, as a thought experiment, that the only remaining runs are of exactly 2×10^{10} coins each, and that it is our turn. If the number of remaining runs is odd, we can move in a way that destroys one of the runs, and then the AI will do the same, and so on until we leave the AI with no moves. If the number of remaining runs is even, we can take the first 10^{10} coins from one run, leaving the other 10^{10} behind as a smaller run. Now the AI is in the same bad "even number of remaining runs" situation that we were just in.

We can set up a situation like this by making many moves early on that leave behind runs of exactly 2×10^{10} coins. For example, we can repeatedly choose the largest remaining run of size 3×10^{10} or greater, and start from the $(2 \times 10^{10} + 1)$ -th coin from the left in that group, leaving a run of exactly size 2×10^{10} behind (in addition to any leftover piece to the right of our move). Then, as long as there are still runs larger than 2×10^{10} but smaller than 3×10^{10} , we can use our moves to destroy them by moving in their centers. Our goal is to eliminate all runs larger than 2×10^{10} before the AI randomly destroys all of our runs of 2×10^{10} . Intuitively, we do not need to worry too much about this, since the AI is more likely to move within some remaining larger runs than within our last remaining run of 2×10^{10} runs, so it will usually be helping us out! Once all runs are of size 2×10^{10} or smaller, and we have at least one run of size 2×10^{10} , we have imposed the near-lockdown situation described above.

We hope you will forgive us for not trying to calculate an exact success probability for the above strategy, but one can use a local simulation to show that it wins, e.g., all 100000 of 100000 games. The chances of it losing more than one of 500 games are vanishingly small... and even if the worst somehow happens, in this case, we do have some control over the overall randomness, and we can try again with a slight tweak.

Other parity-based strategies

In Round 1C this year, we had [Bacterial Tactics](#), another problem about a turn-taking game. You may recall that the analysis brought up the [Sprague-Grundy theorem](#); can we use a similar parity-based strategy here?

As in our 2×10^{10} strategy, we can try to keep the number of remaining runs even for the AI by moving in the middle of a run if there is an even number of runs, or otherwise taking the left end of some other large run. Empirically, this strategy solves TS1, and it even solves TS2 if we always take the largest remaining run. It ends up being similar to our best strategy described above, even though it does not allow for such fine control.

We can even achieve a perfect solution for this problem by exhaustively calculating Grundy numbers and storing them using run-length encoding! However, it is possible to arrive at the 2×10^{10} idea above, which is good enough, without knowing anything about this theory.